

Desenvolvimento Full Stack

Introdução ao Node.js

1. O que é o Node.js?

O **Node.js** é uma plataforma open-source baseada no mecanismo V8 do Google Chrome que permite executar código JavaScript no lado do servidor.

Ele permite a construção de aplicações rápidas e escaláveis, tornando-o muito popular para desenvolvimento web.

2. Principais Características:

Event-driven: Utiliza um modelo orientado a eventos, o que significa que as operações I/O (leitura de arquivos, consultas a banco de dados) não bloqueiam o fluxo da aplicação.

Single-threaded com Non-blocking I/O: O Node.js utiliza uma única thread para gerenciar diversas conexões simultaneamente, economizando recursos.

Ecossistema NPM: Possui o Node Package Manager (NPM), o maior repositório de pacotes de código aberto do mundo.

3. Casos de Uso do Node.js:

Aplicações web em tempo real, como chats.

APIs RESTful.

Aplicativos baseados em microserviços.

Introdução ao Sequelize

1. O que é o Sequelize?

O **Sequelize** é uma biblioteca ORM (Object-Relational Mapping) para Node.js. . Ele simplifica o processo de comunicação com bancos de dados relacionais, como MySQL, PostgreSQL, SQLite e MariaDB.

Permite que você trabalhe com tabelas do banco de dados como se fossem objetos do seu código.

2. Principais Funcionalidades:

- **Modelos:** Estruturas que representam tabelas do banco.
- **Migrações:** Controle e versionamento das alterações na estrutura do banco.
- **Associações:** Definição de relacionamentos entre tabelas (1:N, N:N).
- **Consultas:** Operações de CRUD (Create, Read, Update, Delete) simplificadas.

3. Por que usar o Sequelize?

Reduz a necessidade de escrever queries SQL diretamente.

Facilita a manutenção do código.

Torna o código mais legível e alinhado ao paradigma orientado a objetos.

Como o Node.js e Sequelize Funcionam Juntos

Node.js como Base:

- O Node.js é usado como o ambiente para criar uma aplicação backend, processar requisições e enviar respostas.
- Ele oferece módulos (como o `express`) para criar servidores HTTP e gerenciar rotas.

Sequelize no Backend:

- O Sequelize é integrado ao Node.js para fazer operações com o banco de dados.
- Ele usa os modelos para mapear tabelas e permite realizar consultas e alterações diretamente no banco.

Fluxo de Dados:

- O cliente (frontend) envia uma solicitação para o servidor (Node.js).
- O servidor processa essa solicitação e usa o Sequelize para buscar ou modificar dados no banco.
- O Sequelize retorna os dados ou resultados, que o servidor envia de volta ao cliente.

Benefícios dessa Integração

Produtividade:

Redução de código repetitivo (queries SQL).

Uso de pacotes NPM para adicionar funcionalidades.

Manutenção:

Estrutura organizada com o Sequelize usando `models` e migrações.

Facilidade para integrar novos desenvolvedores ao projeto.

Flexibilidade:

O Node.js e o Sequelize suportam múltiplos bancos de dados.

Node.js permite integrar ferramentas modernas, como WebSocket.

Conceito de ORM

O **Mapeamento Objeto-Relacional (ORM)** é uma técnica que conecta paradigmas de programação orientada a objetos com bancos de dados relacionais. Ele permite que desenvolvedores interajam com tabelas de banco de dados como se fossem objetos no código, abstraindo operações SQL complexas.

Objetivos principais do ORM:

- Reduzir a necessidade de escrever SQL manual.
- Simplificar o gerenciamento de dados em aplicações.
- Aumentar a produtividade do desenvolvedor.

Funcionamento do ORM

Um ORM funciona transformando classes e objetos da programação em estruturas do banco de dados e vice-versa. Ele realiza automaticamente operações como inserir, atualizar, deletar ou buscar dados.

Exemplo:

- Uma classe `Usuario` no código se mapeia a uma tabela `Usuarios` no banco.
- Propriedades como `Nome` e `Email` se tornam colunas.

Processo básico:

1. O desenvolvedor cria modelos (classes) que representam tabelas.
2. O ORM interpreta essas classes e cria as tabelas no banco de dados (se configurado).
3. Operações no banco são feitas como métodos no código, como `Save()`, `Find()`, etc.

Benefícios do ORM

1. Abstração do Banco de Dados:

- Não é necessário conhecimento avançado de SQL.
- Foco na lógica da aplicação, enquanto o ORM trata a comunicação com o banco.

2. Produtividade:

- Menos código para escrever e manter.
- Operações CRUD (Criar, Ler, Atualizar, Deletar) são simplificadas.

3. Portabilidade:

- Muitos ORMs suportam múltiplos bancos de dados (ex.: MySQL, PostgreSQL, SQLite).
- Trocar de banco de dados exige poucas ou nenhuma mudança no código.

4. Manutenção:

- Alterações nas tabelas são feitas diretamente nos modelos.
- Fácil de entender para novos desenvolvedores.

Limitações do ORM

1. Desempenho:

- Em operações muito complexas, ORMs podem ser menos eficientes que consultas SQL escritas à mão.

2. Curva de Aprendizado:

- Entender conceitos como migrações e configurações pode ser desafiador para iniciantes.

3. Abstração Excessiva:

- Pode dificultar a depuração de problemas diretamente no banco de dados.

Principais Ferramentas e Bibliotecas ORM

1. Hibernate (Java):

- Mais usado em projetos Java.
- Suporte a diversas funções avançadas, como cache e consultas HQL.

2. Entity Framework (C#):

- ORM oficial para a plataforma .NET.
- Integração fácil com outras ferramentas da Microsoft.

3. Sequelize (Node.js):

- Simples de configurar e usar.
- Suporte a vários bancos de dados como MySQL, PostgreSQL, e SQLite.

4. SQLAlchemy (Python):

- Oferece controle tanto em alto nível quanto em baixo nível.
- Flexível para vários tipos de projetos.

Implementação Prática

CRUD com Sequelize

Pré-requisitos

Antes de iniciar, precisamos de algumas ferramentas básicas:

1. **Instalar o Node.js:** Baixe e instale o Node.js a partir de nodejs.org.
2. **Ter um banco de dados configurado:** Usaremos o MySQL (outros bancos também funcionam, como PostgreSQL ou SQLite). Instale o MySQL ou use ferramentas como o XAMPP.

Etapa 1: Configurando o Ambiente

1. **Crie uma pasta para o projeto:** Abra o terminal e crie uma pasta para o seu projeto:

```
mkdir projeto-crud  
cd projeto-crud
```

2. **Inicie o projeto com o npm:** Este comando cria o arquivo `package.json`:

```
npm init -y
```

3. **Instale as dependências:** Instale o Sequelize, o CLI do Sequelize e o driver do MySQL:

```
npm install sequelize sequelize-cli mysql2
```

4. **Inicialize o Sequelize:** Gere a estrutura inicial do projeto:

```
npx sequelize-cli init
```

Serão criadas as pastas:

- `config/` (para as configurações do banco de dados),
- `models/` (para os modelos),
- `migrations/` (para as migrações),
- `seeders/` (para dados de teste).

Etapa 2: Configurando o Banco de Dados

1. **Edite o arquivo de configuração:** No arquivo `config/config.json`, insira os dados do seu banco:

```
{
  "development": {
    "username": "root",
    "password": "sua_senha",
    "database": "projeto_crud",
    "host": "127.0.0.1",
    "dialect": "mysql"
  }
}
```

Substitua `root` e `sua_senha` pelas informações do seu banco.

2. **Crie o banco de dados:** Execute o comando para criar o banco no MySQL:

```
npx sequelize-cli db:create
```

Etapa 3: Criando o Modelo e a Migração

1. **Gere um modelo:** Crie o modelo `Usuario` com os campos: `nome (string)` e `email (string)`

```
npx sequelize-cli model:generate --name Usuario --attributes
nome:string,email:string
```

2. **Verifique a migração:** No arquivo gerado em `migrations/`, você verá algo assim:

```

module.exports = {
  up: async (queryInterface, Sequelize) => {
    await queryInterface.createTable('Usuarios', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      nome: {
        type: Sequelize.STRING,
        allowNull: false
      },
      email: {
        type: Sequelize.STRING,
        allowNull: false
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  down: async (queryInterface, Sequelize) => {
    await queryInterface.dropTable('Usuarios');
  }
};

```

3. **Execute a migração:** Crie a tabela no banco com o comando:

```
npx sequelize-cli db:migrate
```

Etapa 4: Criando o CRUD

1. **Adicione o arquivo de conexão ao banco:** Crie um arquivo `db.js` na raiz do projeto para conectar-se ao banco:

```

const { Sequelize } = require('sequelize');
const sequelize = new Sequelize('projeto_crud', 'root', 'sua_senha', {
  host: '127.0.0.1',
  dialect: 'mysql'
});

module.exports = sequelize;

```

2. Crie o arquivo do CRUD: No arquivo `index.js`, escreva as operações CRUD:

```
const sequelize = require('./db');
const { DataTypes } = require('sequelize');

// Definir modelo
const Usuario = sequelize.define('Usuario', {
  nome: {
    type: DataTypes.STRING,
    allowNull: false
  },
  email: {
    type: DataTypes.STRING,
    allowNull: false
  }
});

// Sincronizar com o banco
sequelize.sync({ force: false }).then(() => {
  console.log('Banco sincronizado!');
});

// Funções CRUD
async function criarUsuario(nome, email) {
  const usuario = await Usuario.create({ nome, email });
  console.log('Usuário criado:', usuario);
}

async function listarUsuarios() {
  const usuarios = await Usuario.findAll();
  console.log('Lista de usuários:', usuarios);
}

async function atualizarUsuario(id, novosDados) {
  const usuario = await Usuario.findByPk(id);
  if (usuario) {
    await usuario.update(novosDados);
    console.log('Usuário atualizado:', usuario);
  }
}

async function excluirUsuario(id) {
  const usuario = await Usuario.findByPk(id);
  if (usuario) {
    await usuario.destroy();
    console.log('Usuário excluído');
  }
}

// Exemplos de uso
(async () => {
  await criarUsuario('Fabiano', 'fabiano@email.com');
  await listarUsuarios();
  await atualizarUsuario(1, { nome: 'Fabiano Flores' });
  await excluirUsuario(1);
})();
```

Etapa 5: Testando a Aplicação

1. **Execute o projeto:** Rode o arquivo `index.js`:

```
node index.js
```

2. **Verifique o banco:** Após executar, os dados serão inseridos e atualizados no banco. Você pode usar ferramentas como o **MySQL Workbench** para visualizar as tabelas e registros.